DOCKER_SETUP

HelixCode Docker Setup

This document describes the Docker-based setup for HelixCode, providing a complete containerized environment with all components.

Overview

The Docker setup provides: - **Complete Containerization**: All HelixCode components in Docker containers - **Network Access**: REST API accessible from the entire network - **Project Access**: Mounted directories for workspace and projects - **Distributed Processing**: Worker nodes for parallel task execution - **Easy Management**: Facade script for simplified container management

Quick Start

Prerequisites

- Docker and Docker Compose installed
- At least 4GB RAM available
- Git (for cloning the repository)

Installation

1. **Clone the repository** (if not already done):

   ```
   git clone <repository-url>
   cd HelixCode
   ```

2. **Setup environment**:

   ```
   cp .env.example .env
   # Edit .env file with your preferences
   ```

3. **Make the facade script executable**:

   ```
   chmod +x helix
   ```

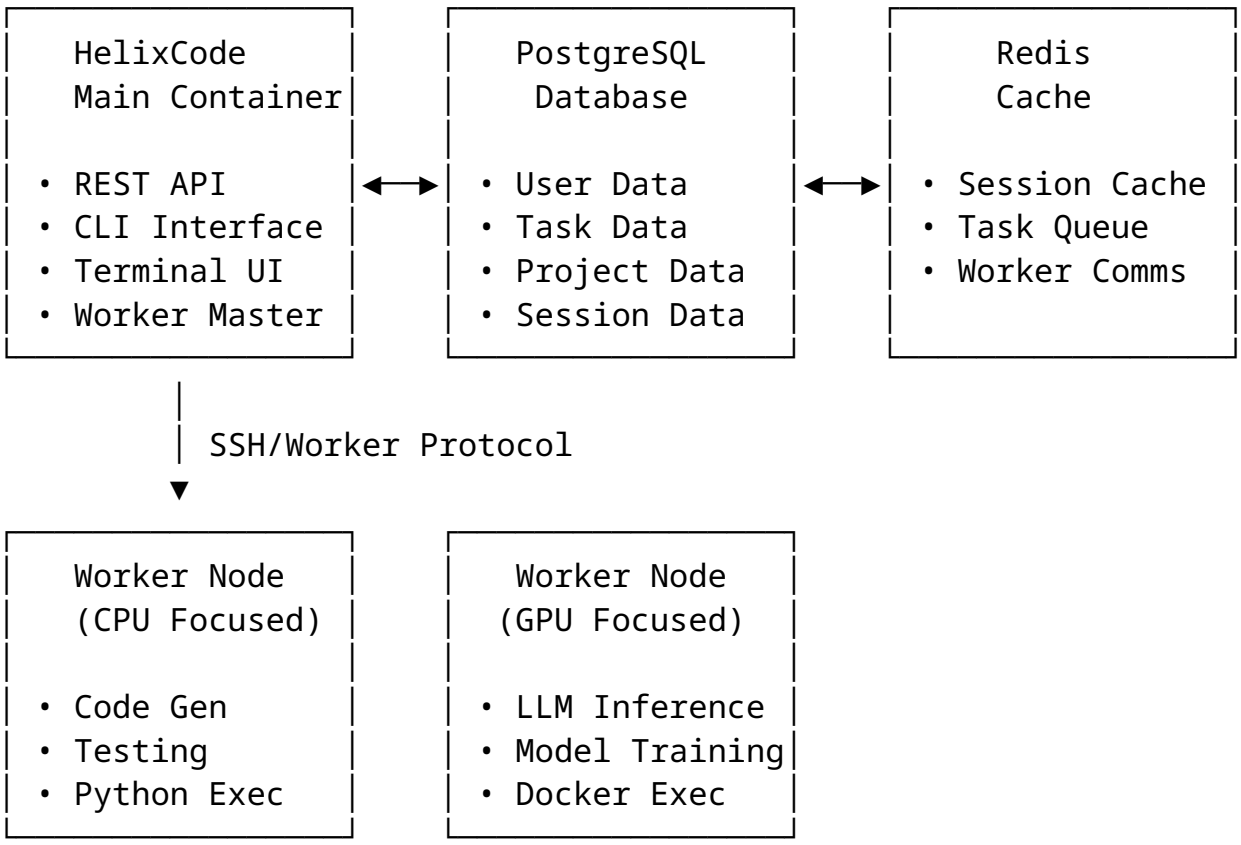
4. **Start the container**:

   ```
   ./helix start
   ```

5. **Access the services**:
    - REST API: http://localhost:8080
    - Terminal UI: `./helix tui`
    - CLI: `./helix cli --help`

Architecture

Container Structure



Network Configuration

- **Internal Network**: `helixcode-network` (172.20.0.0/16)
- **Port Mapping**:
    - 8080: REST API (configurable via HELIX_API_PORT)
    - 2222: SSH Worker Connections (configurable via HELIX_SSH_PORT)
    - 3000: Web Interface (configurable via HELIX_WEB_PORT)

Volume Mounts

- `./workspace`: Main working directory
- `./projects`: Project storage
- `./shared`: Shared configuration and discovery data

- `./helix_code/config`: Application configuration
- `./helix_code/assets`: Static assets

Usage

Using the Facade Script

The `helix` script provides a unified interface:

```
# Start the container
./helix start

# Check status
./helix status

# Run CLI commands
./helix cli --list-workers
./helix cli --health

# Start Terminal UI
./helix tui

# View logs
./helix logs
./helix logs helixcode  # Specific service

# Stop the container
./helix stop

# Restart
./helix restart
```

Direct Container Access

You can also access the container directly:

```
# Execute commands in the container
docker exec -it helixcode helix cli --help

# Access shell
docker exec -it helixcode /bin/bash

# View container processes
docker exec helixcode ps aux
```

Project Development

1. **Mount your projects**:
   - Place your projects in the `./projects` directory
   - They will be accessible inside the container at `/projects`
2. **Use the workspace**:
   - The `./workspace` directory is your main working area
   - All HelixCode operations will use this directory
3. **Access from multiple nodes**:
   - In distributed mode, the shared configuration is available in `./shared`
   - Other nodes can discover and connect automatically

Configuration

Environment Variables

One-step path (recommended): run `./setup.sh`. It creates `.env` from `.env.example` at mode 0600 and generates a unique random value for `HELIX_DATABASE_PASSWORD`, `HELIX_REDIS_PASSWORD` and `HELIX_AUTH_JWT_SECRET`. Re-running it never overwrites a value you already set.

The compose files and the server carry **no** credential of their own (CONST-042): they read these variables from `.env`, and refuse to start with an actionable message if one is missing. `.env` is gitignored and must never be committed.

To do it by hand instead, create a `.env` file in the root directory:

```
# Required for security — use a unique, randomly generated value
        per install,
# e.g. `openssl rand -hex 24`. Never reuse a value published
        anywhere.
HELIX_DATABASE_PASSWORD=your-db-password
HELIX_AUTH_JWT_SECRET=your-jwt-secret
HELIX_REDIS_PASSWORD=your-redis-password

# Port configuration (adjust if ports are occupied)
HELIX_API_PORT=8080
HELIX_SSH_PORT=2222
HELIX_WEB_PORT=3000

# Network mode
HELIX_NETWORK_MODE=standalone  # or 'distributed'
```

```
# Auto-port adjustment
HELIX_AUTO_PORT=false
```

Network Modes

Standalone Mode (Default)

- Single container with all services
- Suitable for development and single-user scenarios
- Simple setup and management

Distributed Mode

- Multiple containers can connect and share workload
- Automatic service discovery
- Suitable for team environments and high-load scenarios
- Enable with: `HELIX_NETWORK_MODE=distributed`

Port Management

If ports are occupied, you can: 1. **Manual adjustment**: Change port numbers in `.env` 2. **Auto-adjustment**: Set `HELIX_AUTO_PORT=true` to automatically find available ports

Advanced Usage

Adding Worker Nodes

Additional worker nodes can be added to the `docker-compose.helix.yml`:

```
worker-3:
  build:
    context: .
    dockerfile: Dockerfile.worker
  container_name: helixcode-worker-3
  hostname: worker-3
  environment:
    - WORKER_ID=worker-3
    - WORKER_CAPABILITIES=data-processing,analysis
    - WORKER_MAX_TASKS=2
    - HELIX_SERVER=helixcode:2222
  volumes:
    - ./helix_code/test/ssh-keys:/root/.ssh:ro
    - ./workspace:/workspace:ro
```

```
    - ./projects:/projects:ro
  depends_on:
    helixcode:
      condition: service_healthy
  networks:
    - helixcode-network
```

Custom Worker Images

Create custom worker images with specific capabilities:

```
FROM helixcode/worker:latest

# Install additional dependencies
RUN apk add --no-cache python3 py3-pip
RUN pip3 install numpy pandas scikit-learn

# Set custom capabilities
ENV WORKER_CAPABILITIES=data-science,machine-learning,python
ENV WORKER_MAX_TASKS=3
```

External Services

To use external PostgreSQL or Redis instead of the included ones:

```
# In .env file
HELIX_DATABASE_URL=postgres://user:pass@host:port/database?
        sslmode=disable
HELIX_REDIS_URL=redis://user:pass@host:port/database

# Then comment out postgres and redis services in docker-
        compose.helix.yml
```

Troubleshooting

Common Issues

1. **Port conflicts**:

   ```
   # Check what's using the ports
   netstat -tulpn | grep :8080
   netstat -tulpn | grep :2222
   netstat -tulpn | grep :3000
   ```

```
# Or use auto-port adjustment
HELIX_AUTO_PORT=true ./helix start
```

2. **Container not starting**:

```
# Check logs
./helix logs

# Check Docker daemon
docker info

# Check available resources
docker system df
```

3. **Worker connection issues**:

```
# Check SSH keys
ls -la helix_code/test/ssh-keys/

# Test worker connectivity
docker exec helixcode ssh -o StrictHostKeyChecking=no
        worker-1 echo "test"
```

Resource Requirements

- **Minimum**: 2GB RAM, 2 CPU cores
- **Recommended**: 4GB RAM, 4 CPU cores
- **Production**: 8GB+ RAM, 8+ CPU cores

Performance Optimization

1. **Increase worker resources**:

```
deploy:
  resources:
    limits:
      cpus: '4.0'
      memory: 8G
```

2. **Use external databases** for production workloads

3. **Enable distributed mode** for multi-user scenarios

4. **Mount SSDs** for better I/O performance

Security Considerations

1. **Change default passwords** in `.env`
2. **Use HTTPS** in production (reverse proxy recommended)
3. **Restrict network access** to necessary ports only
4. **Regularly update** container images
5. **Monitor resource usage** and set limits

Development

Building Custom Images

```
# Build main image
docker build -t helixcode:latest .

# Build worker image
docker build -f Dockerfile.worker -t helixcode-worker:latest .
```

Testing Changes

```
# Run tests in container
./helix exec go test ./...

# Development mode with mounted source
./helix exec bash -c "cd /app && go run cmd/server/main.go"
```

Support

For issues and questions: 1. Check the logs: `./helix logs` 2. Verify container status: `./helix status` 3. Check resource usage: `docker stats` 4. Review this documentation

License

This Docker setup is part of the HelixCode project. See the main project LICENSE for details.